ORIGINAL ARTICLE

Financial asset price prediction with graph neural network-based temporal deep learning models

Yasin Uygun[1] · Emre Sefer[1]

Received: 30 June 2024 / Accepted: 14 August 2025 / Published online: 8 September 2025
© The Author(s), under exclusive licence to Springer-Verlag London Ltd., part of Springer Nature 2025

Abstract
Changes in the prices of multiple financial assets over time can be characterized by their complex nature and interdependence. More traditional forecasting approaches may overlook the interdependencies among these assets, since they may not fully consider the spatial-temporal dependencies between them. Graph neural networks (GNNs) have emerged as powerful tools for modeling complex relational dependencies in areas such as social network analysis and traffic forecasting. However, their application in asset price prediction remains relatively unexplored. Here, we investigate GNNs' effectiveness in forecasting multiple financial asset prices jointly, specifically in the foreign exchange (Forex) and cryptocurrency markets. We employ three spatio-temporal GNN frameworks-MTGNN, StemGNN, and FourierGNN-which are all recognized for their state-of-the-art performance in forecasting multivariate time series. These models transform time-series data into graphs and capture both spatial and temporal dependencies. They significantly outperform the baseline methods, including LSTM, ARIMA, and VAR, in predicting financial asset prices in the highly volatile cryptocurrency market. While the performance gap is less obvious in the relatively stable Forex market, GNN-based models still demonstrate a general advantage over LSTM, although they are outperformed by ARIMA. Through a series of experiments and backtesting strategies, we assess the predictive power and profitability of these models in portfolio construction. Our code and datasets are publicly available at https://github.com/seferlab/temporal gnn.

Keywords Asset price prediction · Deep learning · Graph neural network · Time series · Forex · Cryptocurrency

1 Introduction

The prediction of asset prices, including stocks, through artificial intelligence-based trading systems has been a subject of study for nearly three decades. In contemporary financial markets, the scope of tradeable instruments extends beyond stocks to include Forex instruments [1], cryptocurrencies [2], options [3], exchange-traded funds (ETFs) [4], commodities [5], etc. Correspondingly, the role of artificial intelligence-based trading systems has grown in significance and functionality across diverse global markets [5].

Deep learning methods have recently exhibited superior performance in various classification and prediction tasks compared to more traditional machine learning models like Support Vector Machines (SVMs). Notably, deep learning excels in image processing tasks such as segmentation and classification, marking a significant departure from conventional methodologies [6–8]. A parallel trend has emerged in financial asset price prediction tasks. While traditional time-series models like long short-term memory (LSTM), convolutional neural networks (CNN), and recurrent neural networks (RNN) have been employed, the application of these techniques in

 https://doi.org/10.1007/s00521-025-11586-8

financial prediction lags behind their prevalence in other artificial intelligence areas such as computer vision [9, 10] and Natural Language Processing [11, 12].

Graphs model a set of items as vertices, and the relationship between them as edges. Graphs have recently attracted more attention due to their significant expressive capabilities. For instance, relationships between humans can be modeled as social networks [13]. Similarly, the relationship between companies' trade relationships can be modeled as a supply chain graph [14]. Graphs are a common non-Euclidean data structure for machine learning problems. To apply deep learning-based techniques to nonlinear graph data, graph neural networks (GNNs) [15] have been developed. The utilization of GNNs in modeling complex data relationships has been showing significant promise across distinct domains [16–19], mainly due to their remarkable performance. Historically, the application of GNNs in forecasting multivariate time series has been largely confined to specific domains such as power systems [20] and traffic forecasting [17]. However, they haven't been studied extensively for predicting asset prices in financial markets, underscoring the need for further research on the effectiveness of GNNs in capturing intricate financial dependencies.

Financial markets are characterized by their dynamic and interdependent nature, where the price movements of multiple assets can affect each other. For instance, in the Cryptocurrency market, Bitcoin price has a direct effect on the price of alternative cryptocurrencies such as Ethereum, Solana, and Litecoin [21]. Such dependencies also exist in the reverse direction; a surge in Ethereum price may cause sudden decreases in Bitcoin price, especially if the Ethereum price change is due to the movement of Bitcoin investments into Ethereum. In another example, Forex currencies also affect each other. For instance, EUR/USD will increase if the European Central Bank increases the interest rate higher than the United States Central Bank [22]. These examples present a potential use of GNNs, which are capable of capturing the joint patterns of multiple different assets' price movements.

Here, we focus on the application of GNNs, specifically, MTGNN [23], StemGNN [24], and FourierGNN [25], in forecasting the prices and price directions of multiple financial assets jointly, particularly in foreign exchange (Forex) and Cryptocurrency markets. The goal is to evaluate their predictive power in various market conditions, such as bull and bear markets, and assess their profitability through portfolio construction and backtesting [26]. The outcomes of these methods will be compared with those of baseline methods, including LSTM [27, 28], ARIMA [29], and VAR [30], to demonstrate the relative effectiveness of GNNs in financial forecasting.

According to the detailed experiments, the GNN models, particularly MTGNN, outperformed the remaining methods both in terms of MAPE for price prediction and in terms of Accuracy, Precision, Recall, and F_1 score for price direction prediction. While MTGNN's improvement is statistically significant in both Forex and Cryptocurrency markets, the improvement has been even more pronounced in the highly volatile Cryptocurrency market compared to the baseline models. In terms of portfolio-based backtesting performance, MTGNN performs on a par with LSTM in the less volatile Forex market, while generally outperforming the rest of the portfolio strategies even though they did not consistently result in significantly profitable cases in simulated trading scenarios. However, the LSTM-based portfolio strategy significantly outperformed all the other models in the highly volatile Cryptocurrency market. Such portfolio-based results suggest a complex and nonlinear relationship between forecasting accuracy and financial performance: higher accuracy does not necessarily lead to higher financial returns.

Our main contributions can be summarized as follows:

- We show that GNNs can outperform more conventional statistical time-series approaches such as ARIMA [29] and VAR [30], as well as deep learning models such as LSTM [31] in predicting financial asset prices owing to their ability to capture complex inter-series relationships in multi-asset data.
- We expand the application of GNNs to the financial forecasting domain, a domain that is traditionally dominated by more conventional time-series models.
- We show that GNNs can be used to learn latent dependencies among multiple assets, even though multi-asset price data does not have an explicit graph structure.

We structure the remainder of this article as follows: Sect. 1.1 discusses relevant deep learning approaches for time-series forecasting. Next, Sect. 2 presents key spatio-temporal GNN architectures. In Sect. 3, we detail our experimental setup. Section 4 reports our empirical findings and explores their financial implications through backtesting. Finally, Sect. 5 provides concluding remarks.

1.1 Related work

In recent years, the surge in computational capacity has led to the emergence of newer deep learning-based approaches in various fields such as finance, etc. Several of these studies are in finance which is this research's main focus. Existing studies, such as [32–41], have explored deep learning approaches for forecasting stock markets, leveraging events data, financial time-series datasets, and news integration. Relevant work, such as [37, 38], and [42–44] incorporate technical analysis indicators into prediction models, employing feedforward neural networks, CNNs, and transformers with two-dimensional matrix characterizations of technical analysis datasets. Notably, deep learning demonstrates accurate learning and generalization across buy and sell time points over extended testing horizons in these studies.

Asset price prediction over time via deep learning techniques can be seen as an emerging topic. Before the emergence of deep learning-based methods, most of the time-series prediction methods were statistical techniques. The autoregressive integrated moving average (ARIMA) [29], Gaussian Processes (GP) [45], and Vector Auto Regression (VAR) [30] could be seen as statistical methods to model time-series data. While ARIMA and GP are quite common in time-series prediction mainly because of their highly interpretable and simple nature, they do not extend robustly to multivariate time-series data, and they depend strongly on the stationary nature of time-series data. Similarly, although VAR is extensively used for its ability to model the interdependencies among multiple time series, it is inherently linear and may not effectively capture the complex, nonlinear dynamics often present in financial time series.

Deep learning methods can capture the nonlinear nature of the data as well as not rely on the stationarity of the data. The existing time-series deep learning techniques could be divided into 2 parts: 1- Univariate price prediction, where univariate methods analyze each asset's time-series dataset individually without taking into account the dependencies among multiple different assets data [46]. For instance, FC-LSTM [47] predicts the future value of a univariate time series with LSTM and fully connected layers. On the other hand, SMF [48] enhances the LSTM model by partitioning the cell states of a given series into a collection of distinct frequency parts via the Discrete Fourier Transform. Lastly, N-BEATS [49] comes up with a deep learning system for the basis expansion of univariate time-series data via multiple fully connected layers. 2- Multivariate price prediction which treats a set of multiple time series as a joint entity [50–52]. For instance, TCN [53] applies dilated convolutions to model high-dimensional time-series data. LSTNet [54] integrates both recurrent neural network (RNN) and convolutional neural network (CNN) to infer local dependencies between multiple time-series datasets. CNNs summarize the interactions between multiple variables into a global latent state, so they won't be able to exploit the hidden dependencies among variable pairs.

Graph neural networks (GNNs) are quite successful in modeling spatial correlations between network vertices for various tasks including graph classification, vertex clustering, edge inference, and vertex classification [55]. By assuming that node states depend on those of their neighbors, GNNs learn high-level vertex embeddings by aggregating information from the vertex's neighbors [56–58]. They encompass various architectures that leverage different neural network techniques, namely, recurrent GNNs, convolutional GNNs, graph autoencoders, and spatio-temporal GNNs as categorized in [59]. Specifically, spatio-temporal GNNs handle multivariate time-series data with dependency graphs describing relationships between variables. These models combine temporal modeling (via RNN [60, 61] or CNN [62, 63]) with spatial modeling through graph convolution.

Among the existing spatio-temporal GNNs, DCRNN [61] integrates spatial and temporal relations in the convolutional recurrent neural network to predict traffic. ST-GCN [64] integrates gated temporal convolution and

graph convolution via spatio-temporal convolutional blocks. GraphWaveNet [65] captures spatio-temporal relationships by combining graph convolution with dilated causal convolutions.

Several spatio-temporal GNNs have also been developed specifically for financial forecasting. HATS [66] uses predefined corporate relational data and a hierarchical graph attention network to predict stock movement directions. THGNN [67] infers dynamic relationships among asset prices by combining a transformer encoder with temporal and heterogeneous graph attention. DySTAGE [68] converts asset pricing time series into dynamic graphs to capture evolving correlations over time.

More recent approaches include MTGNN, StemGNN and FourierGNN, all specifically designed for multivariate time-series data by transforming them into graphs and capturing both spatial and temporal dependencies. MTGNN [23] utilizes a graph learning module to infer unidirectional correlations between variables, and to model temporal and spatial dependencies, it has novel dilated inception and mix-hop propagation layers. On the other hand, StemGNN [24] captures multivariate and temporal relationships jointly in the spectral domain. Meanwhile, FourierGNN [25] introduces a novel hypervariate graph data structure, which treats each series value as a node in fully connected graphs within lookback windows, thereby allowing it to model spatio-temporal dynamics in an integrated manner while operating in the Fourier space.

2 Multivariate financial asset price prediction by GNNs

Graph neural networks (GNNs) [15] operate directly on graphs, utilizing their connectivity structure and node features to learn a representation of the data. This capability has driven a surge in the application of GNNs across diverse fields, including social network analysis and protein–protein interaction modeling [44, 69–72], where the graph structure is typically predefined. Moreover, different spatio-temporal GNN architectures were proposed to handle data where the graph structure is latent. This includes multivariate time series where dependencies among variables can be modeled as an implicit graph.

The forthcoming sections explain how such spatio-temporal architectures model the multivariate time-series data. We begin by revisiting the foundational version of GNNs [15], referred to here as the vanilla GNN. Subsequently, we detail three spatio-temporal architectures-MTGNN [23], StemGNN [24], and FourierGNN [25]-all of which are subjects of our financial analysis.

2.1 Vanilla GNN

The vanilla GNN is designed to capture the state embeddings of nodes, representing the information of a node's local neighborhood in a graph. The learning process for these embeddings is structured as follows:

- *Initialization:* Each node is initialized with a state derived from its features.
- *State Update:* The state of a node is iteratively updated through a *local transition function* f_W, which incorporates the node's features as well as the features and states of its adjacent nodes. This function f_W is consistent across all nodes.
- *Output Computation:* After many iterations, which allow the node states to converge, an *output function* g is applied to the state of each node to generate an output value. This value is then utilized for various downstream tasks, such as node classification or regression.
- *Training:* The model is trained by adjusting the parameters of f_W and g to minimize the loss function. For supervised learning tasks, an example loss function is typically a sum of the squared differences between the predicted and target values for each node, as shown below:

$$\text{Loss} = \sum_{i=1}^{p} (t_i - o_i)^2 \tag{1}$$

In Equation 1, p represents the number of nodes that have supervised target information. For each node i, t_i denotes the target value, which could be continuous in regression tasks. The term o_i represents the output of the GNN for node i, as computed by the output function g.

Figure 1 depicts node 1 and its neighborhood. The shaded area represents the aggregation process where the information from neighboring nodes and the edges connecting to node 1 are considered to update its state. Here, node 1's state is updated by passing the features of the node, along with those of its adjacent nodes and edges, into f_W.

The vanilla GNN model is a foundational approach that paved the way for more complex GNN architectures. It provides a structured way to learn how to encode graph topology into state embeddings of nodes, which can then be used for various predictive and classification tasks. Although it excels at handling relational dependencies, it is designed to work with a well-defined graph structure, making it less suitable for scenarios where the graph structure is unknown beforehand, like in the case of multivariate time-series data. This problem led to the development of GNN frameworks that can learn graphs from latent relationships between individual time series such as MTGNN [23], StemGNN [24], and FourierGNN [25].

2.2 MTGNN

Multivariate time series forecasting with graph neural networks (MTGNN) is a deep learning framework specifically designed to forecast multivariate time series without predefined graph structures [23]. It treats each individual time series as a node in a graph, with edges representing hidden dependencies and interactions. This approach allows MTGNN to learn the graph structure directly from the data, by uncovering latent relationships. Unlike traditional models, MTGNN integrates graph neural networks (GNNs) with temporal convolutional networks (TCNs) to capture both spatial and temporal dependencies simultaneously. In essence, MTGNN figures out connections by watching the data and learning patterns. It understands time patterns, like whether something follows daily or weekly cycles. Then, it combines this understanding of connections between things and changes over time to make better predictions. So, MTGNN is like a very attentive observer that learns both what changes, and how different things affect each other in order to make accurate forecasts. Analogously, assume that there is a big city with bus stops where buses run from one stop to another, and the city operates on a schedule (i.e., data changing over time). Our goal will be to predict traffic at each bus stop in the near future, using traffic at nearby stops as well as how traffic patterns change over time.

The model combines graph structure learning and graph neural network training into a single end-to-end process. This ensures that the learned graph is optimized specifically for the forecasting task. Its key components are: 1- Dynamic Graph Generation Module: MTGNN learns an adaptive adjacency matrix from data using a self-attention mechanism. This helps it to capture dynamic, potentially non-Euclidean relationships among

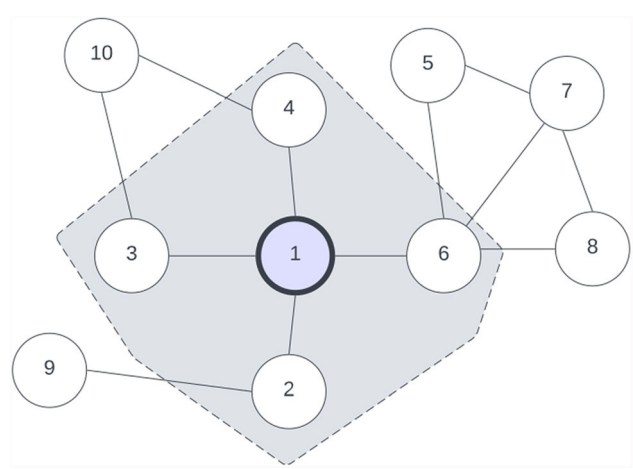

Fig. 1 The state of node 1 is updated based on its own features and the aggregated features of its neighbors. (Adapted from Scarselli et al. [15])

variables (distinct time-series). 2- Graph Convolution Module: MTGNN applies graph convolutions using both the learned graph and an optional static graph. It uses a dual graph convolution mechanism to better capture both local and global variable relationships. 3- Temporal Convolution Module: MTGNN applies dilated causal convolutions to model long-term temporal dependencies efficiently. 4- Gated Skip Connection: MTGNN combines GNN and TCN outputs using skip connections and gated mechanisms, improving gradient flow and model expressiveness.

As depicted in Fig. 2, MTGNN comprises three core components: graph learning layer, graph convolution (GC) modules, and temporal convolution (TC) modules.

Initially, the graph learning layer initializes random node embeddings and learnable model parameters. The TC and GC modules, interleaved within the architecture, incorporate residual and skip connections to prevent the vanishing gradient problem. Throughout each epoch, these modules process the node embeddings, thereby capturing temporal and spatial dependencies. Subsequently, the model calculates the stochastic gradient from the loss, which is then used to update both the model parameters and the node embeddings.

The graph learning layer constructs an adjacency matrix to capture the complex relationships within the multivariate time-series data. It updates this matrix using the node embeddings across epochs. One challenge is that as the number of time series increases, the computational cost for calculating similarities between pairs of nodes rises quadratically. To mitigate this problem, the layer samples the top-k closest nodes as neighbors for each node, thereby sparsifying the adjacency matrix. Moreover, this layer learns unidirectional relationships within the multivariate time-series data, acknowledging that influences among series can be asymmetric. In the adjacency matrix A it constructs, for any positive entry A_{ij}, indicating an influence from series i to series j, the corresponding entry A_{ji} is set to zero.

The graph convolution modules exploit spatial dependencies encapsulated in the graph structure using two novel mix-hop propagation layers. Mix-hop propagation enables the learning about neighbors at different hops through the employment of higher-order message passing, which allows for longer-range interactions [73]. However, aggregating neighborhood information on different degrees of nodes can potentially introduce noise. MTGNN's novelty in mix-hop propagation Lies in maintaining the original states of the nodes. In turn, this approach helps preserve local information as the nodes learn from a deep neighborhood. For this, the layers employ two steps: 1- recursively propagating node information throughout the graph to capture a deep neighborhood, and 2- selectively filtering the gathered information at each hop to prevent the propagation of noise and mitigate the over-smoothing that often accompanies an increase in the number of hops.

The temporal convolution modules capture temporal dependencies by employing a series of modified 1D convolutional filters. These modules consist of two dilated inception layers, which incorporate dilated convolutions to extend the range of temporal patterns the model can capture without increasing its depth or the size of its filters. Here, choosing an appropriate kernel size poses a challenge; larger kernels might miss subtle short-term patterns, whereas smaller ones might not adequately capture longer-term sequences. MTGNN addresses this dilemma through an inception strategy, which involves concatenating the outputs from a diverse array of kernel

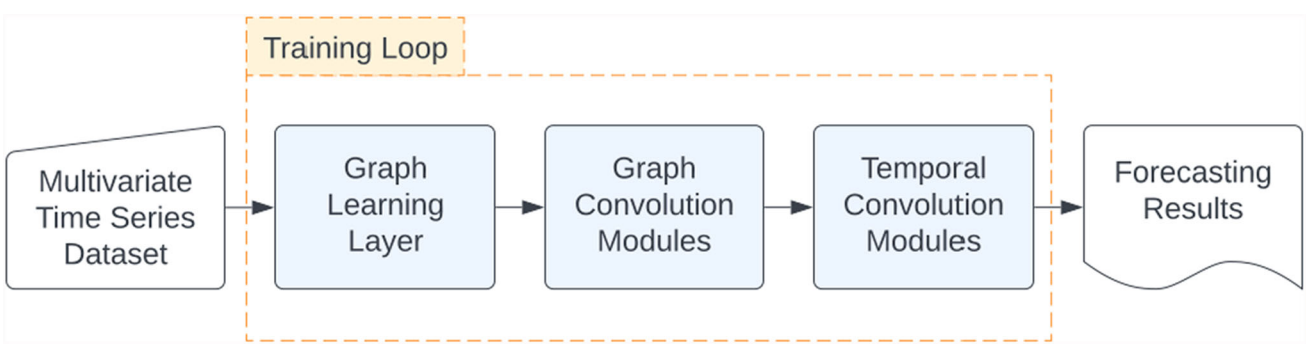

Fig. 2 Simplified MTGNN Workflow (Adapted from Wu et al. [23])

sizes. Specifically, it utilizes filters of sizes 1×2, 1×3, 1×6, and 1×7, a configuration to support a range of periodicities characteristic of temporal data-such as weekly and monthly cycles-are effectively represented. By adopting this approach, MTGNN aims to balance granularity with the ability to recognize and integrate long-term temporal sequences, thus enhancing its forecasting capabilities for complex time-series data. This end-to-end process iteratively refines the model, culminating in the generation of forecasting results.

2.3 StemGNN

Spectral Temporal Graph Neural Network (StemGNN) presents a different approach to multivariate time-series forecasting [24]. Similar to MTGNN, it simultaneously captures intra-series temporal correlations and inter-series relationships without requiring predefined graph topologies. Diverging from MTGNN and similar models, StemGNN operates within the spectral domain. It utilizes the Graph Fourier Transform (GFT) to learn inter-series correlations and the Discrete Fourier Transform (DFT) to learn temporal dependencies, integrating these processes into a unified end-to-end framework. This spectral domain focus offers enhanced clarity in the spectral representations of data. Put simply, StemGNN is a smart forecasting tool that is similar to MTGNN in terms of modeling change over time and connections. Apart from financial forecasting, its other example usages include traffic at different locations, power usage across a city, or air pollution in various neighborhoods.

Figure 3 illustrates the StemGNN process, starting with the input of multivariate time-series data, X. StemGNN converts this input into a graph, learns relationships from this graph, and generates forecasts.

The initial step involves the latent correlation layer, whose task is uncovering the underlying relationships among time-series variables. This layer treats each time series as a distinct graph node and seeks to learn the edges that represent the interdependencies between them. While such edges can sometimes be predefined based on domain knowledge, StemGNN provides a way to learn these relationships directly from the data. It utilizes a Gated Recurrent Unit (GRU) to capture temporal features at each timestamp t, and then employs a self-attention mechanism over the GRU's final hidden state to capture the latent correlations between node pairs. The resulting weight matrix serves as an adjacency matrix, encapsulating the strength of connections in the resultant graph. The resulting graph, G, is then processed through two successive StemGNN blocks, interconnected by skip connections to allow building deeper models.

StemGNN blocks aim to model the spatial and temporal dependencies. Each of them begins with the application of GFT to convert the graph into a spectral matrix representation. Following this, the Spectral

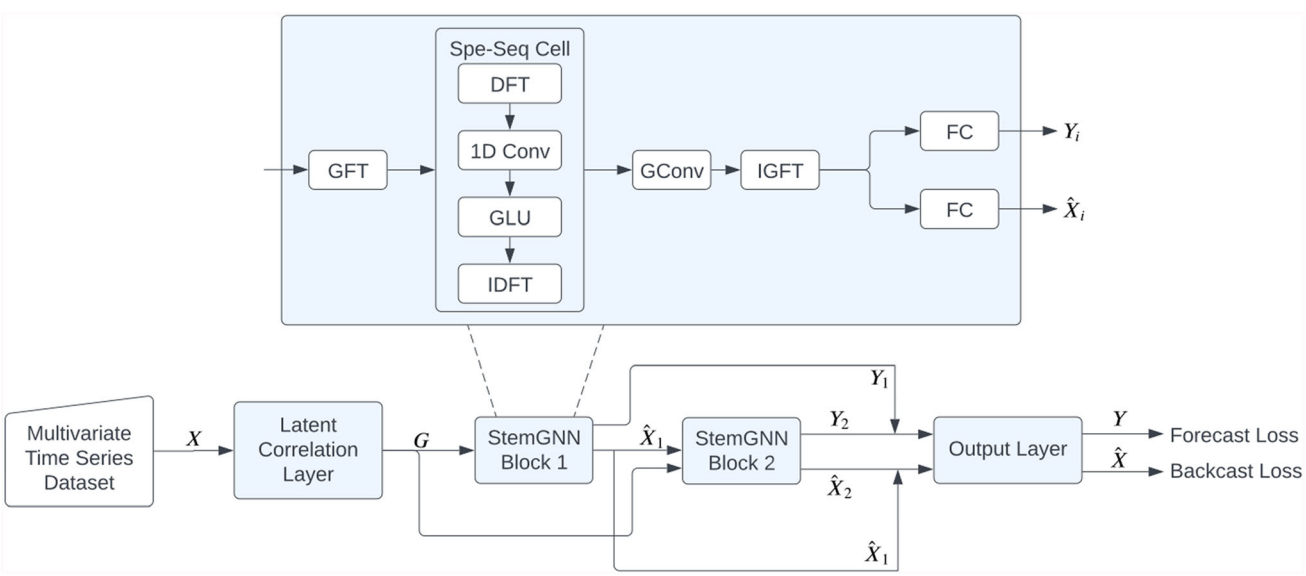

Fig. 3 Overview of StemGNN Architecture (Adapted from Cao et al. [24])

Sequential (Spe-Seq) Cell employs DFT for temporal dependency analysis, transforming time-domain signals into the frequency domain. Subsequently, the combination of 1D convolution and Gated Linear Units (GLU) facilitates learning temporal feature representations in the frequency domain. These are then reconverted into the time domain using the Inverse DFT (IDFT). The Spectral Graph Convolution (GConv) in each block captures spatial dependencies through graph convolution filtering of the spectral representation, followed by the Inverse GFT (IGFT) to revert to the time-series representation in the original space.

Finally, the network's output layer-composed of GLU and fully connected sub-layers-generates two types of outputs: forecasting outputs, Y, for predicting future values, and backcasting outputs, $\hat{X}$, for reconstructing past values. The final loss function of the network is derived from the combined losses of these outputs. Overall, StemGNN's architecture offers a comprehensive method for jointly addressing spatial and temporal dependencies in multivariate time series within the spectral domain.

2.4 FourierGNN

Fourier Graph Neural Network (FourierGNN) deals with the multivariate time-series forecasting from a purely graph-based perspective [25]. Unlike most spatio-temporal GNNs that decouple spatial and temporal modeling through distinct modules-as in MTGNN and StemGNN-FourierGNN treats the entire input as a single graph structure. Specifically, it introduces the concept of a hypervariate graph, where each value in a multivariate time-series window is modeled as an individual graph node, interconnected with every other node. This results in a fully connected graph with $N \times T$ nodes for N time series and a window size of T, as illustrated in Fig. 4. In this way, both spatial and temporal dependencies are modeled jointly as edges between nodes.

This formulation avoids the potential incompatibilities introduced by the use of separate network structures in other methods. However, since the hypervariate graph is fully connected, the number of nodes grows quadratically with both the number of series and the window size. To reduce the resulting computational burden, FourierGNN uses Fourier Graph Operators (FGOs). They replace standard graph convolutions, which rely on dense matrix multiplications, with equivalent element-wise operations in the frequency domain. This reduces the complexity to a log-linear scale, thanks to the convolution theorem [74].

The learning process begins by projecting the hypervariate graph into a d-dimensional embedding space. The node embeddings are then transformed into the frequency domain via the Discrete Fourier Transform (DFT), processed by stacked FGO layers with nonlinear activation functions, and finally returned to the time domain

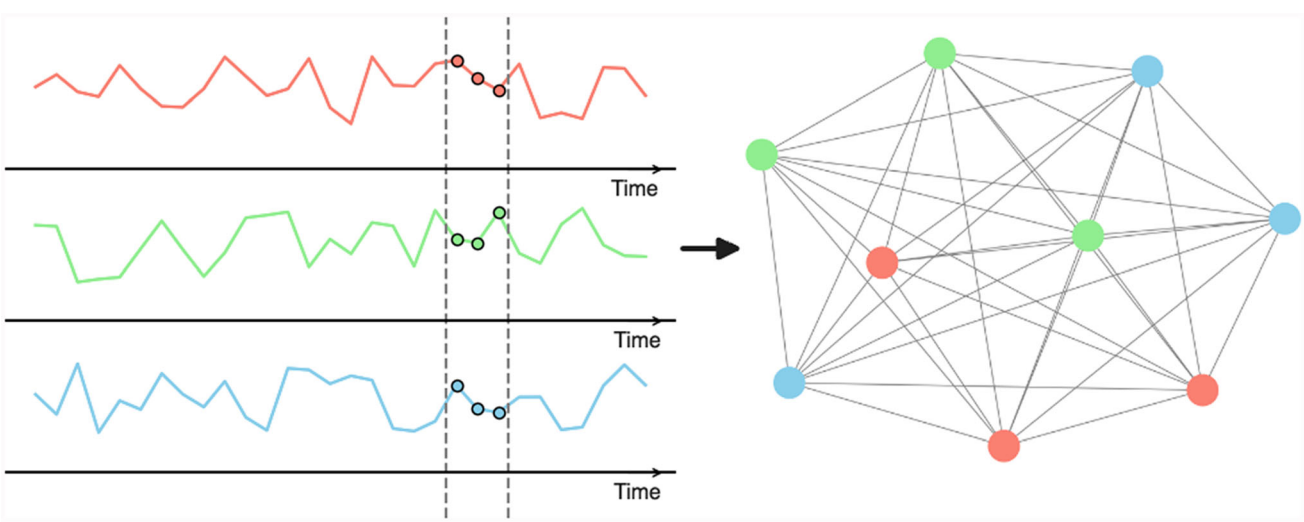

Fig. 4 A hypervariate graph with three time series and a window size of 3. (Adapted from Yi et al. [25])

using the Inverse Discrete Fourier Transform (IDFT). The output is passed through fully connected layers to generate the forecast. Figure 5 provides an overview of this process.

The main differences between FourierGNN, StemGNN, and MTGNN lie in how they handle spatio-temporal dependencies, and especially how they process the temporal component of multivariate time series. FourierGNN has an efficient and expressive temporal representation. Moreover, FourierGNN captures global periodic structures better than sliding window approaches. While StemGNN learns dynamic spatio-temporal relationships instead of relying on static graphs, FourierGNN applies the Discrete Fourier Transform (DFT) on the time dimension to extract frequency-domain patterns. In practice, this allows the model to capture periodicities and long-range dependencies more effectively than time-domain methods. As a result, FourierGNN's integrated approach with hypervariate graphs in the Fourier domain enables it to model spatio-temporal dependencies with lower computational cost compared to architectures that utilize classical graph convolutions.

2.5 Spatio-temporal GNNs for multivariate time-series data

Ultimately, spatio-temporal GNNs emerge as powerful forecasting tools, particularly well suited for intricate and interdependent domains such as financial markets. Their strength lies in their ability to construct representations for each individual time series that go beyond merely encoding historical values, as they effectively capture both spatial and temporal dependencies. For example, in financial forecasting, GNNs can adeptly capture how changes in one asset's performance relate to changes in others. These characteristics position them as a viable choice for modeling complex multivariate financial assets.

From the computational complexity perspective, FourierGNN uses DFT for temporal modeling where the cost per-node is $O(T \log T)$, and it is $O(NT \log T)$ for N nodes. StemGNN uses DFT like FourierGNN, but incorporates a recurrent memory mechanism. DFT cost is $O(NT \log T)$. Its memory update cost is $O(NTd^2)$ for hidden state dimension d which results in total temporal complexity of $O(NT \log T + NTd^2)$. MTGNN uses dilated temporal convolutions. For L layers with kernel size k, this costs $O(NTkL)$. In terms of space complexity, FourierGNN uses static graphs with standard GCNs or ChebNets, which cost $O(Ed)$ or $O(N^2d)$ for dense graphs. StemGNN learns dynamic adjacency matrices during training, whose cost is $O(N^2d)$ due to graph learning and spatial message passing. Lastly, MTGNN learns adjacency using attention mechanisms, which cost $O(N^2d)$ in general; can be reduced to $O(Nkd)$ using top-k sparsification.

Having established the theoretical foundations and architectural distinctions of the GNN models MTGNN, StemGNN, and FourierGNN, used in this study, we now proceed to evaluate their practical utility in real-world financial forecasting. To this end, we design a comprehensive experimental setup aimed at benchmarking these spatio-temporal GNNs against both statistical and deep learning baselines across two diverse financial domains: the volatile cryptocurrency market and the more stable Forex market. The following section details the datasets, preprocessing steps, hyperparameter tuning, and performance evaluation protocols used to assess the models' predictive accuracy and financial relevance.

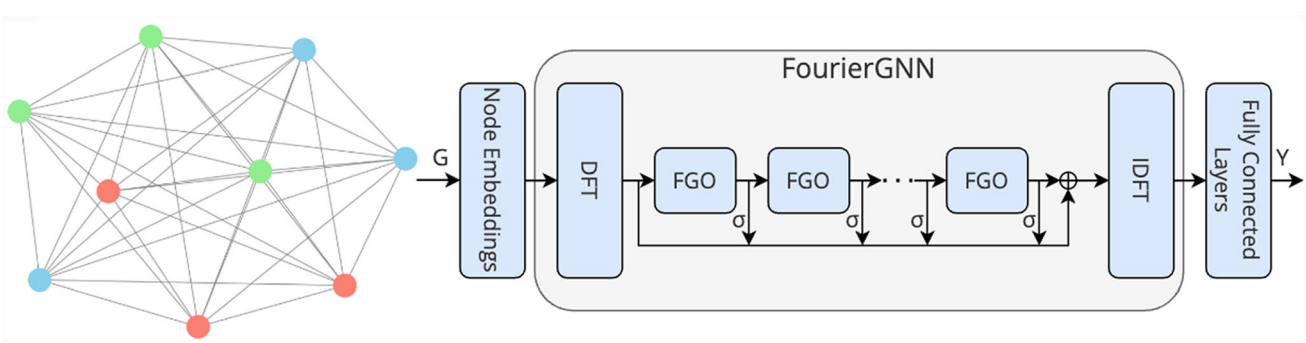

Fig. 5 FourierGNN Architecture Overview (Adapted from Yi et al. [25])

3 Experimental setup

3.1 Dataset description and preparation

The study utilizes two distinct datasets to evaluate the efficacy of GNNs in varying market conditions: Cryptocurrency and Foreign Exchange (Forex) datasets. We obtained the price time series of both datasets by using the yfinance library [75]. The Cryptocurrency dataset comprises daily closing prices from October 2016 to October 2022 for 20 assets selected from the 50 highest market capitalization cryptocurrencies, based on data availability. This dataset includes FIL-USD, ETH-USD, BTC-USD, and 17 other assets. The Forex dataset includes daily closing prices from May 2006 to October 2023 for 10 major currency pairs. The selected Forex pairs are: AUD-USD, TRY-USD, EUR-USD, and 7 other pairs. For both datasets, included assets are shown in Table 1. Compared to cryptocurrencies, the Forex market is characterized by relatively lower volatility.

Augmented Dickey–Fuller (ADF) tests [76] applied to both datasets indicated non-stationarity in the raw price data. To address this, we transformed both price datasets into return ratios RR_t, calculated by using the following formula:

$$RR_t = \frac{P_t}{P_{t-h}} \tag{2}$$

where P_t represents the price at time t, and h denotes the horizon. For the Forex market, the horizon is set to 5 days to align with its weekday-only trading, while for cryptocurrencies, a 7-day horizon is used, reflecting their trading on every day. This transformation aims to stabilize the mean and variance, rendering the series more suitable for predictive modeling [31, 48]. Return ratio transformation statistically represents the multiplicative factor by which the price changes. It has a number of advantages over log-return transformation: 1- It has an intuitive interpretation, where a return ratio of 1.05, for instance, clearly means a 5% increase in price. It directly reflects relative price movement. 2- It preserves multiplicative structure, which is useful in models where compounding or product-based effects are important (e.g., portfolio growth). 3- It avoids log artifacts on some datasets, which may contain zero or negative prices, where log-returns fail but ratios still work. 4- It is more precise over small windows [77–79]. Subsequently, the return ratios are normalized by dividing each value by the maximum absolute value across the series:

$$NormRR_t = \frac{RR_t}{\max_i |RR_i|}, \quad \forall i \in [1, T] \tag{3}$$

where T is the final time step in a 1-based indexed series. Following this transformation, the normalized return ratios ($NormRR_t$) of all assets' close prices were merged into a multivariate time series. Note that some assets, such as FIL-USD or MKR-USD, have less historical data compared to others like ETC-USD or BTC-USD. To handle this discrepancy, the unavailable prices were backfilled. We acknowledge that this decision to backfill may introduce potential biases into the dataset. However, discarding the initial days with missing data would reduce the dataset's size, thereby limiting the learning potential of the models due to fewer samples. Notably, the

Table 1 Cryptocurrency and Forex assets used in our experiments

Asset type	Used assets
Cryptocurrencies	FIL-USD, MKR-USD, USDT-USD, TRX-USD, ETH-USD, XMR-USD, EOS-USD, XTZ-USD, ETC-USD, MIOTA-USD, XRP-USD, ADA-USD, XLM-USD, DOGE-USD, MANA-USD, LINK-USD, BCH-USD, BNB-USD, BTC-USD, LTC-USD
Forex	AUD-USD, TRY-USD, RUB-USD, INR-USD, EUR-USD, GBP-USD, CHF-USD, CAD-USD, CNY-USD, JPY-USD

backfilled values account for approximately 15% of the cryptocurrency dataset, while the Forex dataset contains no missing values. To assess the impact of backfilling, we also conducted experiments in which we dropped the corresponding unavailable entries. These resulted in marginally worse performance, suggesting that the backfilling strategy does not significantly distort the overall results. Thus, the experimental results reported in this article are based on the backfilled version of the cryptocurrency dataset.

For the purpose of price prediction, we applied a sliding window approach to form the input variable. Each window comprised a sequence of prices $[P_{t-s+1}, P_{t-s+2}, \ldots, P_t]$, which was used to forecast the target price P_{t+h}, where s is a hyperparameter representing the sequence length. In terms of a deep learning baseline, we used long short-term memory (LSTM) which serves as the baseline model in this study.

3.2 Hyperparameter selection

To enable the models for optimal performance, hyperparameter optimization was performed using the Hyperopt [80] library in Python. For deep learning models, the best hyperparameters were determined based on the lowest validation loss, whereas Akaike Information Criterion (AIC) was used for the ARIMA and VAR models. Table 2 presents the selected hyperparameters for each model across the Forex and Cryptocurrency datasets.

Table 2 Best hyperparameters for each model on Forex and Cryptocurrency datasets. The parameter names are kept consistent with those used in the code implementations of the models

Model	Hyperparameter	Forex	Crypto
FourierGNN	Sequence length	15	7
	Embed size	32	8
	Hidden size	64	128
	Learning rate	9.59×10^{-4}	1.15×10^{-4}
	Exponential decay step	5	8
	Decay rate	0.8	0.9
MTGNN	Sequence length	10	35
	Layers	3	3
	Conv channels	4	4
	Learning rate	1.42×10^{-4}	6.35×10^{-4}
	Weight decay	3.49×10^{-5}	2.03×10^{-5}
StemGNN	Sequence length	15	7
	Multi-layer	13	8
	Learning rate	0.01	0.001
	Exponential decay step	13	8
	Decay rate	0.5	0.5
	Dropout rate	0.75	0.5
LSTM	Sequence length	200	7
	Hidden size	16	128
	Num layers	5	3
	Dropout	0.2866	0.3446
	Learning rate	0.0080	0.0069
ARIMA	Autoregressive order (p)	5	5
	Differencing order (d)	1	1
	Moving average order (q)	3	2
VAR	Lag order (p)	20	20

3.3 Performance evaluation

We assess and compare the performance of spatio-temporal GNNs and baseline models via traditional machine learning metrics and financial metrics. In terms of traditional metrics, we use Accuracy, Precision, Recall, and F_1 score to evaluate direction-based decisions such as buy, sell, and hold. These metrics are defined below:

$$\text{Accuracy} = \frac{\sum_{i=1}^{C} TP_i}{\sum_{i=1}^{C}(TP_i + TN_i + FP_i + FN_i)} \tag{4}$$

$$\text{Precision} = \frac{\sum_{i=1}^{C}(\text{Precision}_i \times \text{Support}_i)}{\sum_{i=1}^{C} \text{Support}_i} \tag{5}$$

$$\text{Recall} = \frac{\sum_{i=1}^{C}(\text{Recall}_i \times \text{Support}_i)}{\sum_{i=1}^{C} \text{Support}_i} \tag{6}$$

$$\text{F}_1\text{Score} = \frac{\sum_{i=1}^{C}(\text{F}_1\text{Score}_i \times \text{Support}_i)}{\sum_{i=1}^{C} \text{Support}_i} \tag{7}$$

where C represents the total number of classes, which is 3 in our case for buy, hold, and sell. For each class i, the terms TP_i, TN_i, FP_i, and FN_i denote the counts of true positives, true negatives, false positives, and false negatives, respectively. The term Support_i indicates the size of class i. The precision, recall, and F_1 score for each class i are calculated as:

$$\text{Precision}_i = \frac{TP_i}{TP_i + FP_i} \tag{8}$$

$$\text{Recall}_i = \frac{TP_i}{TP_i + FN_i} \tag{9}$$

$$\text{F}_1\text{Score}_i = \frac{2 \times \text{Precision}_i \times \text{Recall}_i}{\text{Precision}_i + \text{Recall}_i} \tag{10}$$

We evaluate the price prediction performance by Mean Absolute Percentage Error (MAPE), Mean Absolute Error (MAE), Root Mean Square Error (RMSE), and a20-index (A20) which are defined as follows:

$$\text{MAPE} = \frac{1}{n}\sum_{t=1}^{n} \frac{|A_t - F_t|}{|A_t|} \tag{11}$$

$$\text{MAE} = \frac{1}{n}\sum_{t=1}^{n} |A_t - F_t| \tag{12}$$

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{t=1}^{n}(A_t - F_t)^2} \tag{13}$$

$$\text{A20} = \frac{1}{n}\sum_{t=1}^{n}\begin{cases} 1, & \text{if } \dfrac{|A_t - F_t|}{|A_t|} \leq 0.2 \\ 0, & \text{otherwise} \end{cases} \tag{14}$$

where A_t represents the actual value and F_t is our forecast.

Lastly, by comparing their predictive performance measures across ten different training sessions, we calculated p-values to evaluate the statistical significance of the models' prediction performance.

3.3.1 Portfolio metrics

In addition to the traditional machine learning metrics, we evaluate the financial performance of the models. The backtesting training protocol, depicted in Fig. 6, starts with a hyperparameter search step where all available time-series data, excluding the final 104 weeks is used. This data is partitioned into a training set comprising 75% and a validation set forming the remaining 25%. This step selects the best-performing hyperparameter set as described in Sect. 3.2. Subsequently, the model undergoes retraining on progressively diminishing subsets of the trailing data. In each iteration, the model is trained on the available data. To mitigate the risk of overfitting, training incorporates an early stopping mechanism using the validation set, halting when the validation loss ceases to improve. Next, the best model parameters corresponding to the minimum validation loss are restored and employed to predict the forthcoming week's return ratios. This approach emulates a rolling window that moves forward one week at a time, where each week the amount of data excluded from the end of the series diminishes by one week, until all 104 weeks are consumed.

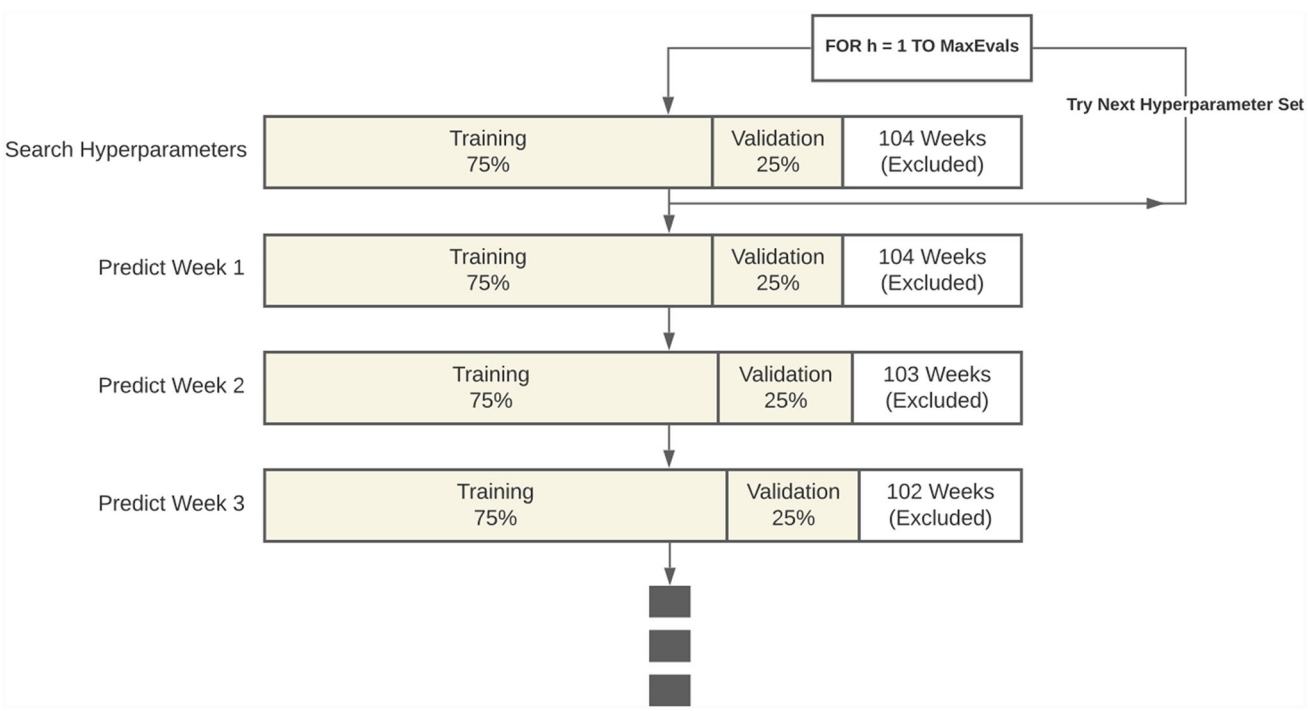

Fig. 6 Data partitioning during backtesting

An equally-weighted long-short portfolio was constructed based on the models' predictions to simulate real-world trading scenarios. Although this strategy does not reflect the full complexity of optimal portfolio management, it aligns with our primary objective: isolating the predictive utility of each model in comparison to GNN-based approaches without the confounding effects introduced by advanced portfolio optimization techniques. Nonetheless, the same model predictions could be employed in more sophisticated trading frameworks-such as end-to-end pipelines that jointly learn forecasting and portfolio allocation [81]. Accordingly, we assume an initial capital of \$100 and apply a trading commission of 0.5% per transaction, given that we trade only liquid Forex and Cryptocurrency names. We evaluate the portfolio performance using the Annualized Return, Annualized Volatility, Sharpe Ratio (*SR*), and Max Drawdown (*MD*). Let P_i denote the vector of the last closing prices of the assets for the i-th week (e.g., Friday for the Forex market, Sunday for the Cryptocurrency market), w_j represent the equal weight of the j-th asset, N be the number of invested assets, Σ be the covariance matrix of asset returns, $\bar{R}$ be the sample mean of weekly returns, and r_f be the risk-free rate corresponding to the three-month U.S. Treasury Bill rates during the trading period [82]. The portfolio metrics are defined as follows:

$$\text{Weekly Return } (R_i) = \frac{P_i}{P_{i-1}} - 1 \tag{15}$$

$$\text{Asset Weight } (w_j) = \frac{1}{N} \tag{16}$$

$$\text{Annualized Return } (AR) = w^T \bar{R} \times 52 \tag{17}$$

$$\text{Annualized Volatility } (AV) = \sqrt{w^T \Sigma w \times 52} \tag{18}$$

$$\text{Sharpe Ratio } (SR) = \frac{AR - r_f}{AV} \tag{19}$$

$$\text{Maximum Drawdown } (MD) = \frac{LowestValue - PeakValue}{PeakValue} \tag{20}$$

Each model's predictions were utilized to construct a long-short portfolio. This portfolio was rebalanced weekly for 104 weeks, based on the model's latest predictions. For a given asset, a long position was initiated only when the model projected an increase in its price. Conversely, a short position was taken in anticipation of a price decline as predicted by the model. In instances where the model did not forecast a definitive increase or decrease, the portfolio's position in that asset was liquidated, and the investment was reverted to cash. See Fig. 7 for the overview of this process.

In addition to these model-specific portfolios in Fig. 7, two baseline portfolios were created for comparative statistical analysis: 1- A random long-only portfolio, which was rebalanced weekly with randomly selected assets; and 2- an "All-asset Equal Portfolio", which involve equal investment across all assets and had the advantage of incurring no transaction costs since there is no rebalancing.

To evaluate the statistical significance of the results, we compared the weekly profits of all portfolios throughout the 104-week simulation. Subsequently, p-values were calculated for all pairwise portfolio combinations (e.g., Random vs. MTGNN, StemGNN vs. LSTM) to assess significance. To maintain clarity and avoid excessive detail, we omit comparisons between baseline models (e.g., ARIMA vs. VAR).

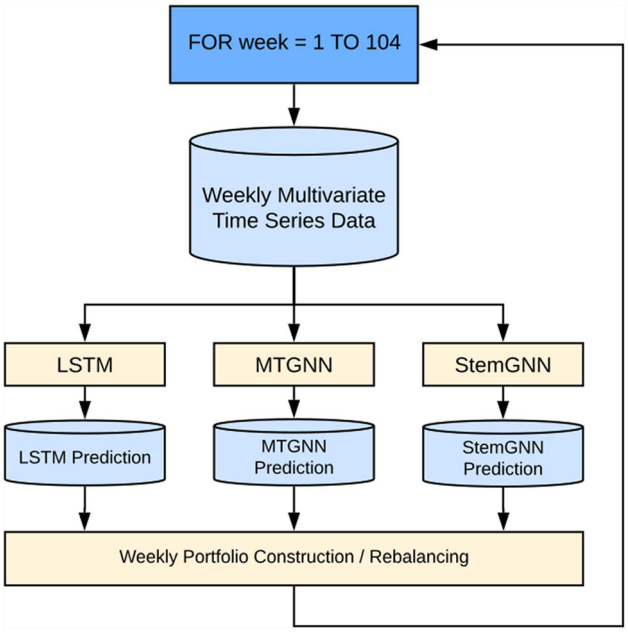

Fig. 7 Overview of the weekly model portfolio rebalancing process

3.4 Implementation details

We used the official codebases for the GNN methods: MTGNN,[1] StemGNN,[2] and FourierGNN.[3] The LSTM baseline was implemented using `PyTorch` [83]. Statistical baselines, ARIMA and VAR, were implemented using the `statsmodels` library [84].

4 Results and discussion

4.1 Comparison via traditional metrics

Figure 8a, b presents the performance of methods on the testing set using MAPE, MAE, RMSE, and A20. The mean and standard deviation are calculated over 10 iterations to account for the stochastic nature of deep learning models. In contrast, ARIMA and VAR are deterministic and thus run only once, resulting in zero standard deviation values in these figures.

The relative standing of the models depends heavily on market volatility. For the highly volatile Cryptocurrency market, all GNN variants significantly outperform the LSTM baseline, with MTGNN delivering the best prediction metrics. Conversely, on the relatively stable Forex market, ARIMA achieves the best overall performance while MTGNN remains the top performer among the deep learning methods. Here, LSTM is slightly ahead of StemGNN and FourierGNN whereas VAR is the weakest model in both markets. It is worth noting that superior predictive performance does not necessarily translate into better financial returns, as different models may have captured more profitable opportunities despite their overall lower predictive performance relative to others. We explore this further in later sections.

[1] https://github.com/nnzhan/MTGNN.
[2] https://github.com/microsoft/StemGNN.
[3] https://github.com/aikunyi/FourierGNN.

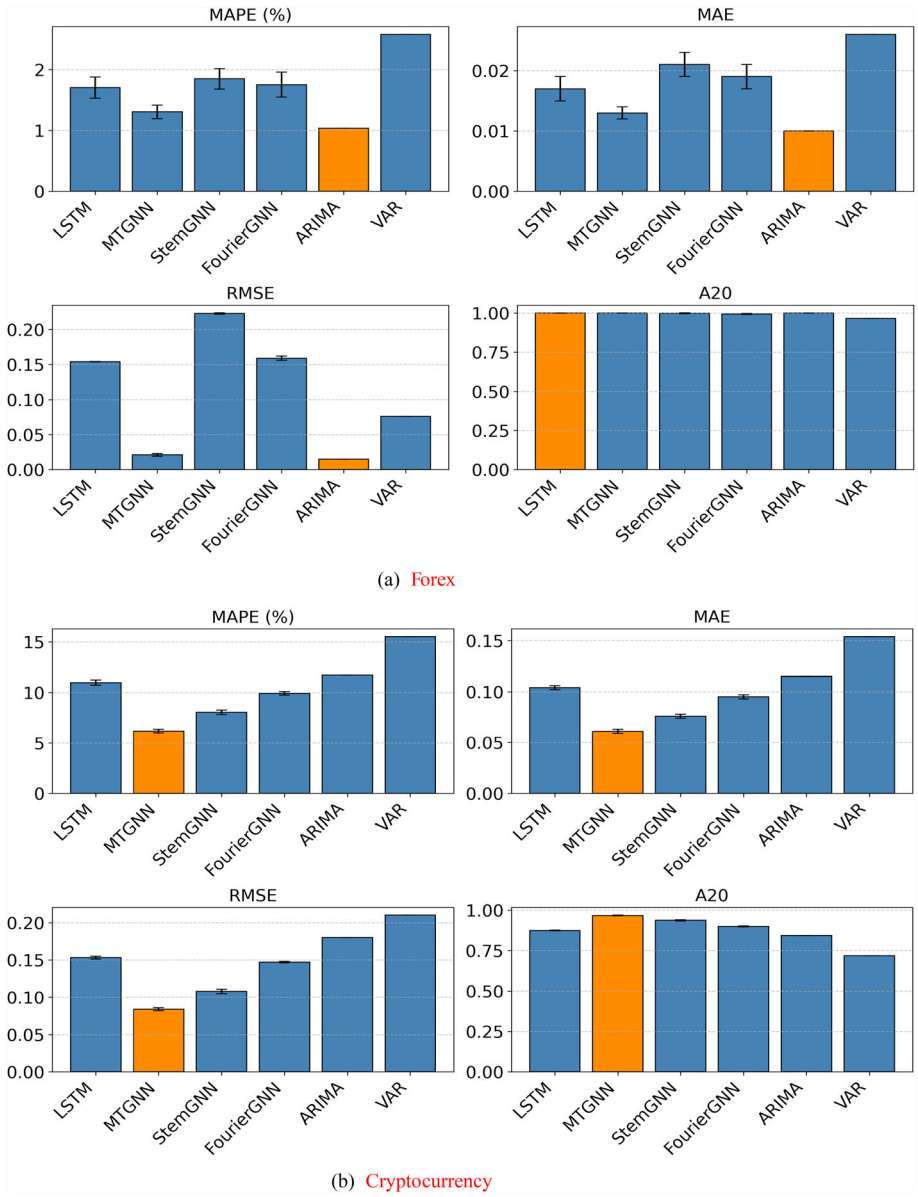

Fig. 8 Prediction metrics of return ratio over 10 iterations. Orange bars indicate top-performing methods (colour figure online)

4.2 Direction-based prediction evaluation

We also evaluate the methods' performance in terms of their ability to predict the direction of price movements. While classifying the price change into "Buy", "Hold", and "Sell" categories, a "Hold" is considered when the price change in either direction remains within 1%. Otherwise, if the price rises by more than 1%, it is labeled as "Buy"; if the price decreases by more than 1%, it receives a "Sell" label. Tables 3 and 4 present the general and category-specific direction-based classification metrics for Forex forecasts, respectively.

According to Table 3, StemGNN and FourierGNN have similar classification metrics while outperforming the remaining methods. This contrasts with the price prediction performance results, where they performed worse than MTGNN, LSTM, and ARIMA. Each method exhibits difficulties with "Buy" and especially "Sell" decisions, while having strong "Hold" values. MTGNN's precision on "Buy" decisions is remarkably better than that of the other models, thereby allowing it to identify more opportunities to enter the market.

Table 3 Classification performance of models for Forex data

Model	Accuracy (%)	Precision (%)	Recall (%)	F_1 (%)
LSTM	57.28	83.15	57.28	67.29
MTGNN	54.95	63.59	54.95	58.57
StemGNN	62.14	**87.79**	62.14	**71.67**
FourierGNN	**62.72**	85.65	**62.72**	71.23
ARIMA	60.29	75.89	60.29	66.36
VAR	60.68	77.32	60.68	67.06

Bold values represent the methods with the best performance

Table 4 The breakdown of Forex Buy/Hold/Sell decision metrics, respectively

Model	Precision (%)	Recall (%)	F_1 (%)
LSTM	4.76/90.44/1.24	23.41/61.12/5.13	7.91/72.95/2.00
MTGNN	**23.81**/78.06/8.07	28.06/64.42/21.31	**25.76**/70.59/11.71
StemGNN	6.93/**95.77**/8.07	30.77/65.21/31.71	11.31/**77.59**/12.87
FourierGNN	6.06/94.36/18.63	**51.85**/65.72/**34.48**	10.85/77.48/24.19
ARIMA	8.66/88.40/**22.98**	36.36/66.75/28.46	13.99/76.06/**25.43**
VAR	11.69/90.28/13.66	30.34/**67.13**/26.51	16.88/77.01/18.03

Bold values represent the methods with the best performance

Table 5 Classification performance of models for Cryptocurrency data

Model	Accuracy (%)	Precision (%)	Recall (%)	F_1 (%)
LSTM	38.88	65.25	38.89	46.99
MTGNN	39.85	42.75	39.85	39.84
StemGNN	**77.52**	**76.82**	**77.52**	**77.08**
FourierGNN	37.57	59.05	37.57	43.37
ARIMA	40.92	59.76	40.92	46.46
VAR	45.53	46.02	45.53	45.63

Bold values represent the methods with the best performance

Table 6 The breakdown of Cryptocurrency Buy/Hold/Sell Decision Metrics, respectively

Model	Precision (%)	Recall (%)	F_1 (%)
LSTM	77.46/24.50/6.03	41.91/20.81/41.48	54.39/22.51/10.54
MTGNN	53.68/34.54/28.13	44.59/18.66/48.69	48.72/24.23/35.66
StemGNN	**80.63**/54.62/**80.71**	**83.86/44.59/81.67**	**82.22/49.10/81.68**
FourierGNN	62.97/**62.25**/6.79	44.66/22.79/46.67	52.26/33.37/11.85
ARIMA	69.99/52.21/10.24	45.34/26.53/45.89	55.03/35.18/16.74
VAR	50.40/44.18/41.27	44.90/43.31/46.99	47.49/43.74/43.95

Bold values represent the methods with the best performance

Similarly, Tables 5 and 6 display these classification metrics for Cryptocurrency forecasts. In the Cryptocurrency market, StemGNN demonstrates significant superiority over the other models in both tables. However, this superiority does not translate into its portfolio performance as detailed in the portfolio analysis.

4.3 Backtesting portfolio analysis

A backtesting of equal-weighted portfolios, each starting with $100, was conducted based on the forecasts provided by the models over a span of 104 weeks in both the Forex and Cryptocurrency markets. To inject a

realistic aspect of trading into the analysis, a transaction cost of 0.5% per trade was factored into the simulation, which is realistic given all names we trade are highly liquid. Figure 9a, b depicts the wealth curves and the trajectories of portfolio capital throughout the backtesting period, offering the investment performance attributed to each method.

Table 7 provides detailed portfolio metrics derived from these trading simulations in Forex market. For the cryptocurrencies, Tables 8 and 9 offer the same metrics, categorizing them into bull and bear market periods, respectively. These periods are assumed based on the observed trends in Cryptocurrency prices, as depicted for various cryptocurrencies in Fig. 10 over the simulation period from October 2020 to October 2022. The first year is characterized by a consistent upward trend, assumed to be a bull market, while the second year shows a consistent downward trend, indicative of a bear market.

Within these tables, return and volatility columns represent annualized return and volatility, respectively, whereas SR and MD define Sharpe Ratio and Max Drawdown respectively. The capital changes throughout the backtesting suggest a disconnection between prediction accuracy and trading performance. In the Cryptocurrency market, LSTM's higher error rate compared to GNN methods did not preclude it from realizing the highest portfolio growth. Similarly, being the best model in all direction-based prediction metrics did not guarantee the highest profit for StemGNN. FourierGNN unexpectedly loses 86% in the bull period. In the ensuing bear period, by contrast, FourierGNN becomes the only strategy that finishes with a positive return and the smallest drawdown, yet its earlier losses leave its terminal wealth well below the other methods. ARIMA and VAR rank mid-pack in the bull run, while ARIMA has the most negative returns in the bear period. In the Forex market,

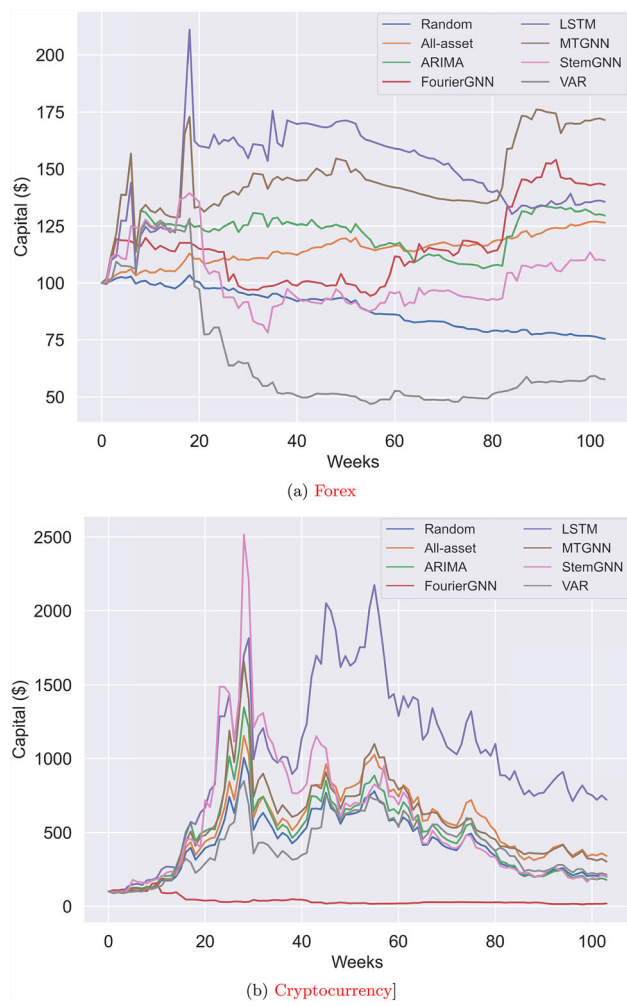

Fig. 9 Wealth curves for tested time-series GNN-based approaches

Table 7 Backtesting performance for Forex portfolios

Portfolio	Return (%)	Volatility (%)	SR	MD (%)
LSTM	25.24	44.44	0.46	−38.33
MTGNN	35.70	40.61	0.76	−27.49
StemGNN	9.52	30.78	0.15	−43.93
FourierGNN	20.34	21.48	0.73	−21.33
ARIMA	18.16	31.14	0.43	−32.11
VAR	−22.16	32.02	−0.84	−63.37
Random	−14.00	7.22	−2.60	−27.12
All-asset	12.12	7.25	1.01	−4.95

Table 8 Backtesting performance for Cryptocurrency portfolios during bull market period

Portfolio	Return (%)	Volatility (%)	SR	MD (%)
LSTM	363.74	120.69	3.00	−50.68
MTGNN	287.17	127.12	2.25	−63.41
StemGNN	335.72	181.91	1.84	−76.10
FourierGNN	−86.06	117.12	−0.74	−88.49
ARIMA	272.08	120.73	2.24	−65.62
VAR	272.57	126.75	2.14	−62.77
Random	249.44	108.26	2.29	−57.57
All-asset	276.74	108.44	2.54	−55.33

Table 9 Backtesting performance for Cryptocurrency portfolios during bear market period

Portfolio	Return (%)	Volatility (%)	SR	MD (%)
LSTM	−70.60	61.27	−1.17	−67.20
MTGNN	−78.56	54.68	−1.46	−72.44
StemGNN	−95.00	75.28	−1.28	−82.54
FourierGNN	9.39	62.98	0.13	−54.05
ARIMA	−116.25	62.50	−1.88	−79.77
VAR	−89.70	66.82	−1.36	−71.15
Random	−99.74	59.59	−1.69	−74.45
All-asset	−73.25	59.65	−1.25	−82.54

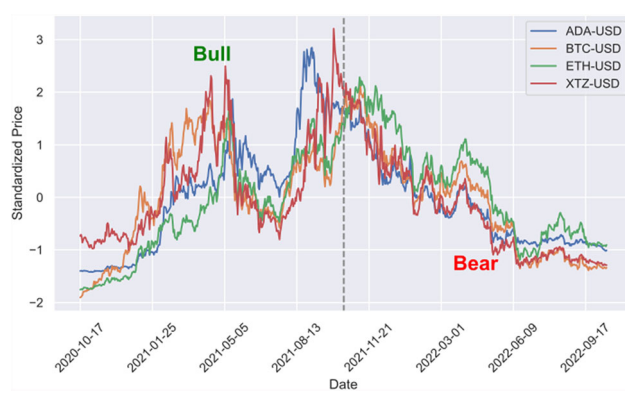

Fig. 10 Assumed bull and bear periods in the Cryptocurrency market

MTGNN model demonstrates the highest annualized return while having high volatility, whereas VAR records the poorest outcome. Notably, the all-asset strategy secures the best risk-adjusted performance and the shallowest drawdown.

4.4 Statistical significance analysis

The statistical significance of the results is analyzed in two parts: 1- the significance of prediction performance is assessed through two-sided independent *t*-tests comparing model prediction metrics; and 2- the significance of portfolio performance is4 evaluated through two-sided paired *t*-tests on the wealth curves of different portfolios across the 104-week simulation period. Tables 10-11 present these analyses for the Forex and Cryptocurrency datasets, respectively. A performance comparison between methods is considered to have a statistically significant difference if the *p*-value is less than 0.05.

For the *t*-tests, a negative *t*-statistic indicates that the model listed first (or on the left side of a comparison pair) has a lower sample mean. In Table 10, this implies better performance for error metrics-namely MAPE, MAE, and RMSE-but worse performance for A20, which is a measure of predictive accuracy. Similarly, in Table 11, which pertains to portfolio capital, negative *t*-statistic values also reflect poorer performance.

In the calculation of *t*-statistic, the denominator involves the standard deviation of the paired differences. When this variance is zero, the statistic becomes undefined or infinite due to division by zero. This issue arises in certain comparisons reported in Table 10. Notably, the comparisons of MTGNN against LSTM and ARIMA in the Forex market using the A20 metric yielded NaN (Not a Number) values for both the *t*-statistic and *p*-value. This occurred because both methods produced identical A20 values-all equal to 1-by closely predicting the actual prices. As a result, the paired differences between the methods were all zero, leading to zero variance and thus rendering the paired *t*-test undefined. Additionally, the comparison between MTGNN and VAR resulted in a *t*-statistic of ∞. In this case, while the paired differences were non-zero (each equal to 0.034, as VAR's A20 values were uniformly 0.966), their variance was still zero, again resulting in a division by zero.

The analysis of prediction errors shows a clear advantage for GNNs over all methods in the cryptocurrency market, whereas statistical methods are more favorable in the Forex market. Specifically, GNN methods outperform LSTM in nearly all cases across both markets. Notable exceptions appear in the Forex market, where LSTM outperforms StemGNN and FourierGNN-showing a significant lead in some metrics and marginal improvements in others. Additionally, MTGNN consistently achieves statistically significantly lower prediction errors than both StemGNN and FourierGNN, establishing it as the best-performing GNN model in terms of predictive performance. In contrast, among statistical forecasting approaches, ARIMA demonstrates significantly better predictive performance than all GNN-based methods in the low-volatility Forex market. While the reason for this discrepancy is difficult to pinpoint, a plausible explanation lies in the interplay between market characteristics and model capacity. The higher volatility and complexity of the Cryptocurrency market may benefit from the greater expressive power of GNNs. However, their expressiveness may also make it more difficult to tune them without overfitting for the relatively stable and low-volatility Forex market when compared to simpler statistical methods like ARIMA. This suggests that the predictive performance of GNNs makes them more favorable in settings with higher volatility and asset interdependence, where their expressiveness is better aligned with the underlying market dynamics.

On the other hand, the portfolio performance analysis presents a more nuanced picture. Within the Forex market, LSTM and MTGNN exhibit comparable performance, both outperforming the other models. Moreover, the All-asset strategy significantly outperforms StemGNN and slightly surpasses FourierGNN in portfolio returns. Consistent with the prediction error analysis-which indicated LSTM's marginal advantage over StemGNN and FourierGNN-LSTM also significantly outperforms these models in portfolio performance. Furthermore, VAR performs significantly worse than all GNN methods in terms of portfolio returns, whereas ARIMA yields better performance than FourierGNN but underperforms relative to MTGNN and StemGNN. In the Cryptocurrency

Table 10 Statistical significance of MAPE, MAE, RMSE, and A20 comparisons in Forex and Cryptocurrency markets

		Forex		**Cryptocurrency**	
Metric	Comparison	*t*-stat	*p*-value	*t*-stat	*p*-value
MAPE	MTGNN vs StemGNN	−8.61 *	$<10^{-5}$ *	−20.31 *	$<10^{-5}$ *
	MTGNN vs FourierGNN	−6.11 *	$<10^{-5}$ *	−46.34 *	$<10^{-5}$ *
	MTGNN vs LSTM	−6.11 *	$<10^{-5}$ *	−48.66 *	$<10^{-5}$ *
	MTGNN vs ARIMA	9.48 **	$<10^{-5}$ **	−77.07 *	$<10^{-5}$ *
	MTGNN vs VAR	−36.14 *	$<10^{-5}$ *	−157.78 *	$<10^{-5}$ *
	StemGNN vs FourierGNN	1.14	0.27055	−20.87 *	$<10^{-5}$ *
	StemGNN vs LSTM	1.90	0.07362	−27.61 *	$<10^{-5}$ *
	StemGNN vs ARIMA	16.80 **	$<10^{-5}$ **	−37.95 *	$<10^{-5}$ *
	StemGNN vs VAR	−14.05 *	$<10^{-5}$ *	−105.40 *	$<10^{-5}$ *
	FourierGNN vs LSTM	0.59	0.56422	−10.98 *	$<10^{-5}$ *
	FourierGNN vs ARIMA	12.17 **	$<10^{-5}$ **	−14.80 *	$<10^{-5}$ *
	FourierGNN vs VAR	−12.86 *	$<10^{-5}$ *	−101.54 *	$<10^{-5}$ *
MAE	MTGNN vs StemGNN	−12.17 *	$<10^{-5}$ *	−17.39 *	$<10^{-5}$ *
	MTGNN vs FourierGNN	−7.33 *	$<10^{-5}$ *	−44.85 *	$<10^{-5}$ *
	MTGNN vs LSTM	−5.70 *	0.00002 *	−47.46 *	$<10^{-5}$ *
	MTGNN vs ARIMA	9.60 **	$<10^{-5}$ **	−75.24 *	$<10^{-5}$ *
	MTGNN vs VAR	−35.16 *	$<10^{-5}$ *	−157.70 *	$<10^{-5}$ *
	StemGNN vs FourierGNN	2.65 **	0.01628 **	−24.14 *	$<10^{-5}$ *
	StemGNN vs LSTM	5.22 **	0.00006 **	−30.08 *	$<10^{-5}$ *
	StemGNN vs ARIMA	21.77 **	$<10^{-5}$ **	−46.86 *	$<10^{-5}$ *
	StemGNN vs VAR	−10.09 *	$<10^{-5}$ *	−124.38 *	$<10^{-5}$ *
	FourierGNN vs LSTM	2.02	0.05825	−10.73 *	$<10^{-5}$ *
	FourierGNN vs ARIMA	13.80 **	$<10^{-5}$ **	−21.83 *	$<10^{-5}$ *
	FourierGNN vs VAR	−11.34 *	$<10^{-5}$ *	−123.99 *	$<10^{-5}$ *
RMSE	MTGNN vs StemGNN	−263.10 *	$<10^{-5}$ *	−22.98 *	$<10^{-5}$ *
	MTGNN vs FourierGNN	−117.33 *	$<10^{-5}$ *	−91.70 *	$<10^{-5}$ *
	MTGNN vs LSTM	−174.64 *	$<10^{-5}$ *	−91.07 *	$<10^{-5}$ *
	MTGNN vs ARIMA	8.60 **	$<10^{-5}$ **	−135.53 *	$<10^{-5}$ *
	MTGNN vs VAR	−73.14 *	$<10^{-5}$ *	−225.32 *	$<10^{-5}$ *
	StemGNN vs FourierGNN	69.41 **	$<10^{-5}$ **	−40.33 *	$<10^{-5}$ *
	StemGNN vs LSTM	308.44 **	$<10^{-5}$ **	−44.09 *	$<10^{-5}$ *
	StemGNN vs ARIMA	1206.54 **	$<10^{-5}$ **	−58.89 *	$<10^{-5}$ *
	StemGNN vs VAR	852.57 **	$<10^{-5}$ **	−115.94 *	$<10^{-5}$ *
	FourierGNN vs LSTM	5.38 **	0.00004 **	−9.08 *	$<10^{-5}$ *
	FourierGNN vs ARIMA	159.19 **	$<10^{-5}$ **	−32.43 *	$<10^{-5}$ *
	FourierGNN vs VAR	91.76 **	$<10^{-5}$ **	−158.83 *	$<10^{-5}$ *
A20	MTGNN vs StemGNN	1.76	0.09465	20.02 *	$<10^{-5}$ *
	MTGNN vs FourierGNN	7.99 *	$<10^{-5}$ *	54.83 *	$<10^{-5}$ *
	MTGNN vs LSTM	NaN	NaN	86.37 *	$<10^{-5}$ *
	MTGNN vs ARIMA	NaN	NaN	121.84 *	$<10^{-5}$ *
	MTGNN vs VAR	∞ *	$<10^{-5}$ *	327.19 *	$<10^{-5}$ *
	StemGNN vs FourierGNN	4.01 *	0.00082 *	23.19 *	$<10^{-5}$ *
	StemGNN vs LSTM	−1.76	0.09465	42.39 *	$<10^{-5}$ *
	StemGNN vs ARIMA	−1.76	0.09465	49.45 *	$<10^{-5}$ *

Table 10 (continued)

Metric	Comparison	Forex		Cryptocurrency	
		t-stat	*p*-value	*t*-stat	*p*-value
	StemGNN vs VAR	35.98 *	$<10^{-5}$ *	172.09 *	$<10^{-5}$ *
	FourierGNN vs LSTM	−7.99 **	$<10^{-5}$ **	21.50 *	$<10^{-5}$ *
	FourierGNN vs ARIMA	−7.99 **	$<10^{-5}$ **	27.79 *	$<10^{-5}$ *
	FourierGNN vs VAR	34.22 *	$<10^{-5}$ *	193.32 *	$<10^{-5}$ *

First model is better. **: The second model is better. No asterisk: No significant difference

Table 11 Statistical significance of portfolio weekly capital comparisons in Forex and Cryptocurrency markets

Comparison	Forex		Cryptocurrency	
	t-stat	*p*-value	*t*-stat	*p*-value
Random versus FourierGNN	−11.85 **	$<10^{-5}$ **	16.87 *	$<10^{-5}$ *
Random versus MTGNN	−26.03 **	$<10^{-5}$ **	−14.01 **	$<10^{-5}$ **
Random versus StemGNN	−9.60 **	$<10^{-5}$ **	−5.99 **	$<10^{-5}$ **
All-asset versus FourierGNN	−0.60	0.54757	18.44 *	$<10^{-5}$ *
All-asset versusMTGNN	−27.36 **	$<10^{-5}$ **	−2.63 **	0.00973 **
All-asset versus StemGNN	7.86 *	$<10^{-5}$ *	−1.49	0.14013
ARIMA versus FourierGNN	4.53 *	0.00002 *	15.87 *	$<10^{-5}$ *
ARIMA versus MTGNN	−18.07 **	$<10^{-5}$ **	−12.21 **	$<10^{-5}$ **
ARIMA versus StemGNN	15.57 *	$<10^{-5}$ *	−4.66 **	$<10^{-5}$ **
FourierGNN versus LSTM	−10.87 **	$<10^{-5}$ **	−18.62 **	$<10^{-5}$ **
FourierGNN versus MTGNN	−21.59 **	$<10^{-5}$ **	−16.76 **	$<10^{-5}$ **
FourierGNN versus StemGNN	8.73 *	$<10^{-5}$ *	−12.15 **	$<10^{-5}$ **
FourierGNN versus VAR	16.73 *	$<10^{-5}$ *	−17.28 **	$<10^{-5}$ **
LSTM versus MTGNN	1.29	0.19981	15.57 *	$<10^{-5}$ *
LSTM versus StemGNN	17.23 *	$<10^{-5}$ *	10.66 *	$<10^{-5}$ *
MTGNN versus StemGNN	21.62 *	$<10^{-5}$ *	−0.81	0.41808
MTGNN versus VAR	23.21 *	$<10^{-5}$ *	10.70 *	$<10^{-5}$ *
StemGNN versus VAR	21.90 *	$<10^{-5}$ *	5.32 *	$<10^{-5}$ *

First portfolio is better. **: The second portfolio is better. No asterisk: No significant difference

market, LSTM delivers the strongest portfolio performance among all strategies. MTGNN outperforms the Random, All-asset, ARIMA, and VAR-based strategies, while performing comparably to StemGNN. However, StemGNN fails to surpass the simple all-asset strategy, and FourierGNN ranks lowest in portfolio performance across all models.

These disparities between predictive and portfolio performance imply that statistically significant prediction accuracy does not necessarily translate to more effective portfolio management. Financial markets, especially volatile ones such as cryptocurrencies, are chaotic systems characterized by high-impact tail-risk events, so-called "Black Swans" [85]. A single well- (or poorly-) timed trade during one of these episodes can outweigh the cumulative effect of many routine decisions, allowing a model with lower overall accuracy to outperform a more accurate rival. Consequently, the practical implications of a model's performance, particularly in terms of risk management and return on investment, should also be evaluated carefully alongside statistical findings, and portfolio construction strategies should account for extreme events while leveraging predictive models.

5 Conclusions and future work

In this study, we investigated the effectiveness of graph neural networks (GNNs), specifically MTGNN and StemGNN, in forecasting financial asset prices in comparison to more traditional time-series forecasting models-namely, LSTM, ARIMA and VAR. The study focused on two distinct financial markets: the foreign exchange (Forex) market and the Cryptocurrency market. From a machine learning standpoint, GNN-based models-particularly MTGNN-demonstrated significantly superior forecasting performance over LSTM in both markets, with especially pronounced gains in the highly volatile Cryptocurrency market. In contrast, the ARIMA baseline outperformed the GNN methods in the less volatile Forex market. Moreover, our backtesting results presented a more complex picture. Despite their superior forecasting accuracy, GNN models did not consistently translate this advantage into improved profitability in automated trading simulations. This underscores the complex and nonlinear relationship between forecasting accuracy and financial performance, suggesting that improved accuracy does not necessarily lead to greater financial returns. Overall, this research offers empirical evidence on the potential of GNNs in financial forecasting-especially in volatile Cryptocurrency markets-while enhancing the understanding of the nonlinear relationship between predictive accuracy and trading profitability.

Although the predictive power of spatio-temporal GNNs can enable more accurate forecasts on volatile and correlative financial assets, their direct application to real-world trading systems would benefit from further enhancements. Accordingly, several avenues for future enhancement of this framework can be explored. One potential avenue is the integration of newly emerging transformer-based spatio-temporal models [86, 87], which have shown promise in capturing long-range dependencies in time series. Additionally, approaches for mitigating the effects of market volatility, particularly relevant in cryptocurrency markets, could further improve forecasting robustness. Financial time series are inherently noisy, influenced by a wide array of factors such as politics and social media. As a result, this study's reliance on historical data may not account for future market conditions and may invalidate our findings across different market structures. Moreover, the timing of trading decisions plays a critical role in investment outcomes; models with comparable predictive power may yield disparate portfolio returns due to fortunate timing in key trades. This raises the possibility that observed simulation results are partially driven by such timing effects. To mitigate this, future work could explore a wider range of trading strategies beyond the equally-weighted long-short approach employed in this study. Lastly, extending the analysis to additional financial instruments, such as stocks, ETFs, or commodities, would allow for broader generalization of the proposed framework and its findings. Collectively, these enhancements can facilitate the integration of GNN-based forecasting models into production-grade financial systems, such as reinforcement learning pipelines, risk-aware portfolio optimizers, or real-time trading platforms. When integrating models to production-scale systems, our models can be trained daily or weekly, which will be feasible since our data is daily. Since dataset frequency is daily, latency will not be an issue either.

Acknowledgements We want to thank Ege Oztas, Nurettin Burak Altintas, and Ayberk Akbalik for valuable discussions.

Data Availability Both cryptocurrency and FX price data in this research is publicly available. Our code and datasets are publicly available at https://github.com/seferlab/temporal gnn.

Declarations

Conflict of interest The authors declare that they have no conflict of interest.

References

1. Ayitey Junior M, Appiahene P, Appiah O, Bombie CN (2023) Forex market forecasting using machine learning: systematic literature review and meta-analysis. J Big Data 10(1):9. https://doi.org/10.1186/s40537-022-00676-2

2. Freeda SE, Selvan TCE, Hemanandhini IG (2021) Prediction of bitcoin price using deep learning model. In: 2021 5th international conference on electronics, communication and aerospace technology (ICECA), pp. 1702–1706. https://doi.org/10.1109/ICECA52323.2021.9676048
3. Ge M, Zhou S, Luo S, Tian B (2021) 3d tensor-based deep learning models for predicting option price. In: 2021 international conference on information science and communications technologies (ICISCT), pp. 1–6. IEEE
4. Zheng W (2021) Exchange-traded fund price prediction based on the deep learning model. In: 2021 China automation congress (CAC), pp. 7429–7434. https://doi.org/10.1109/CAC53003.2021.9727762
5. Cavalcante RC, Brasileiro RC, Souza VLF, Nobrega JP, Oliveira ALI (2016) Computational intelligence and financial markets: a survey and future directions. Expert Syst Appl 55:194–211
6. Ozbayoglu AM, Gudelek MU, Sezer OB (2020) Deep learning for financial applications: a survey. Appl Soft Comput 93:106384
7. Patro R, Duggal G, Sefer E, Wang H, Filippova D, Kingsford C (2012) The missing models: a data-driven approach for learning how networks grow. In: Proceedings of the 18th ACM SIGKDD international conference on knowledge discovery and data mining. KDD '12, pp. 42–50. Association for Computing Machinery, New York. https://doi.org/10.1145/2339530.2339541
8. Sefer E, Kingsford C (2011) Metric labeling and semi-metric embedding for protein annotation prediction. In: Bafna V, Sahinalp SC (eds) Research in computational molecular biology. Springer, Berlin, Heidelberg, pp 392–407
9. Simonyan K, Zisserman A (2014) Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556
10. He K, Zhang X, Ren S, Sun J (2015) Deep residual learning for image recognition. CoRR **abs/1512.03385**
11. Devlin J, Chang M-W, Lee K, Toutanova K (2018) Bert: pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805
12. Yang Z, Dai Z, Yang Y, Carbonell J, Salakhutdinov RR, Le QV (2019) Xlnet: generalized autoregressive pretraining for language understanding. Advances in neural information processing systems, 32
13. Sefer E (2022) Mocmin: convex inferring of modular low-rank contact networks over covid diffusion data. Turk J Electr Eng Comput Sci 30(7):2568–2585
14. Wagner SM, Neshat N (2010) Assessing the vulnerability of supply chains using graph theory. Int J Prod Econ 126(1):121–129. https://doi.org/10.1016/j.ijpe.2009.10.007
15. Scarselli F, Gori M, Tsoi AC, Hagenbuchner M, Monfardini G (2008) The graph neural network model. IEEE Trans Neural Netw 20(1):61–80
16. Zhou J, Cui G, Hu S, Zhang Z, Yang C, Liu Z, Wang L, Li C, Sun M (2020) Graph neural networks: a review of methods and applications. AI Open 1:57–81
17. Jiang W, Luo J (2022) Graph neural network for traffic forecasting: a survey. Expert Syst Appl 207:117921
18. Yuan H, Yu H, Gui S, Ji S (2023) Explainability in graph neural networks: a taxonomic survey. IEEE Trans Pattern Anal Mach Intell 45(5):5782–5799. https://doi.org/10.1109/TPAMI.2022.3204236
19. Dwivedi VP, Joshi CK, Luu AT, Laurent T, Bengio Y, Bresson X (2023) Benchmarking graph neural networks. J Mach Learn Res 24(43):1–48
20. Liao W, Bak-Jensen B, Pillai JR, Wang Y, Wang Y (2021) A review of graph neural networks and their applications in power systems. J Mod Power Syst Clean Energy 10(2):345–360
21. Hafner CM (2020) Alternative assets and cryptocurrencies. J Risk Financ Manag 13(1):7
22. Hanisch M (2018) German and euro area economies will benefit from a us interest rate hike in the short term. DIW Wkly Rep 8(12):115–121
23. Wu Z, Pan S, Long G, Jiang J, Chang X, Zhang C (2020) Connecting the dots: Multivariate time series forecasting with graph neural networks. In: Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining, pp. 753–763
24. Cao D, Wang Y, Duan J, Zhang C, Zhu X, Huang C, Tong Y, Xu B, Bai J, Tong J et al (2020) Spectral temporal graph neural network for multivariate time-series forecasting. Adv Neural Inf Process Syst 33:17766–17778
25. Yi K, Zhang Q, Fan W, He H, Hu L, Wang P, An N, Cao L, Niu Z (2023) Fouriergnn: rethinking multivariate time series forecasting from a pure graph perspective. Adv Neural Inf Process Syst 36:69638–69660
26. Arnott RD, Harvey CR, Markowitz H (2018) A backtesting protocol in the era of machine learning. Available at SSRN 3275654
27. Hochreiter S, Schmidhuber J (1997) Long short-term memory. Neural Comput 9(8):1735–1780
28. Sefer E (2022) Biocode: a data-driven procedure to learn the growth of biological networks. IEEE/ACM Trans Comput Biol Bioinf 19(6):3103–3113
29. Box GE, Jenkins GM, Reinsel GC, Ljung GM (2015) Time series analysis: forecasting and control. John Wiley & Sons, Hoboken
30. Zivot E, Wang J (2006) Vector autoregressive models for multivariate time series. Modeling financial time series with S-PLUS®, 385–429
31. Fischer T, Krauss C (2018) Deep learning with long short-term memory networks for financial market predictions. Eur J Oper Res 270(2):654–669

32. Ding X, Zhang Y, Liu T, Duan J (2015) Deep learning for event-driven stock prediction. In: IJCAI, vol. 15, pp. 2327–2333
33. Längkvist M, Karlsson L, Loutfi A (2014) A review of unsupervised feature learning and deep learning for time-series modeling. Pattern Recogn Lett 42:11–24
34. Fischer T, Krauss C (2018) Deep learning with long short-term memory networks for financial market predictions. Eur J Oper Res 270(2):654–669
35. Krauss C, Do XA, Huck N (2017) Deep neural networks, gradient-boosted trees, random forests: statistical arbitrage on the s &p 500. Eur J Oper Res 259(2):689–702
36. Yoshihara A, Fujikawa K, Seki K, Uehara K (2014) Predicting stock market trends by recurrent deep neural networks. In: Pham D-N, Park S-B (eds) PRICAI 2014: trends in artificial intelligence. Springer, Cham, pp 759–769
37. Sezer OB, Ozbayoglu M, Dogdu E (2017) A deep neural-network based stock trading system based on evolutionary optimized technical analysis parameters. Proced Comput Sci 114:473–480
38. Sezer OB, Ozbayoglu AM (2018) Algorithmic financial trading with deep convolutional neural networks: time series to image conversion approach. Appl Soft Comput 70:525–538
39. Patro R, Sefer E, Malin J, Marçais G, Navlakha S, Kingsford C (2011) Parsimonious reconstruction of network evolution. In: Przytycka TM, Sagot M-F (eds) Algorithms in bioinformatics. Springer, Berlin, Heidelberg, pp 237–249
40. Sefer E (2022) ProbC: joint modeling of epigenome and transcriptome effects in 3d genome. BMC Genomics 23(1):287
41. Sefer E (2021) Hi-c interaction graph analysis reveals the impact of histone modifications in chromatin shape. Appl Netw Sci 6(1):54
42. Tuncer T, Kaya U, Sefer E, Alacam O, Hoser T (2022) Asset price and direction prediction via deep 2d transformer and convolutional neural networks. In: Proceedings of the third ACM international conference on AI in finance. ICAIF '22, pp. 79–86. Association for Computing Machinery, New York. https://doi.org/10.1145/3533271.3561738
43. Gezici AHB, Sefer E (2024) Deep transformer-based asset price and direction prediction. IEEE Access 12:24164–24178. https://doi.org/10.1109/ACCESS.2024.3358452
44. Seyhan B, Sefer E (2023) Nft primary sale price and secondary sale prediction via deep learning. In: Proceedings of the fourth ACM international conference on AI in finance, pp. 116–123
45. Frigola R (2015) Bayesian time series learning with Gaussian processes. PhD thesis, University of Cambridge
46. Murphy WMJ, Chen K (2023) Univariate vs multivariate time series forecasting with transformers. https://openreview.net/forum?id=GpW327gxLTF
47. Xingjian S, Chen Z, Wang H, Yeung D-Y, Wong W-K, Woo W-C (2015) Convolutional lstm network: a machine learning approach for precipitation nowcasting. In: Advances in neural information processing systems, pp. 802–810
48. Zhang L, Aggarwal C, Qi G-J (2017) Stock price prediction via discovering multi-frequency trading patterns. In: Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining, pp. 2141–2149
49. Oreshkin BN, Carpov D, Chapados N, Bengio Y (2020) N-beats: neural basis expansion analysis for interpretable time series forecasting. In: International conference on learning representations. https://openreview.net/forum?id=r1ecqn4YwB
50. Salinas D, Flunkert V, Gasthaus J, Januschowski T (2020) Deepar: probabilistic forecasting with autoregressive recurrent networks. Int J Forecast 36(3):1181–1191
51. Guo S, Lin Y, Feng N, Song C, Wan H (2019) Attention based spatial-temporal graph convolutional networks for traffic flow forecasting. In: Proceedings of the AAAI conference on artificial intelligence, vol. 33, pp. 922–929
52. Loukas A, Perraudin N (2019) Stationary time-vertex signal processing. EURASIP J Adv Signal Process 2019(1):1–19
53. Bai S, Kolter JZ, Koltun V (2018) An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. arXiv preprint arXiv:1803.01271
54. Lai G, Chang W-C, Yang Y, Liu H (2018) Modeling long-and short-term temporal patterns with deep neural networks. In: The 41st international ACM SIGIR conference on research & development in information retrieval, pp. 95–104
55. Wu L, Cui P, Pei J, Zhao L, Guo X (2022) Graph neural networks: foundation, frontiers and applications. In: Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining, pp. 4840–4841
56. Gilmer J, Schoenholz SS, Riley PF, Vinyals O, Dahl GE (2017) Neural message passing for quantum chemistry. In: Proceedings of ICML, pp. 1263–1272
57. Gasteiger J, Bojchevski A, Günnemann S (2019) Predict then propagate: graph neural networks meet personalized pagerank. In: International conference on learning representations, https://openreview.net/forum?id=H1gL-2A9Ym
58. Kipf TN, Welling M (2017) Semi-supervised classification with graph convolutional networks. In: International conference on learning representations. https://openreview.net/forum?id=SJU4ayYgl
59. Wu Z, Pan S, Chen F, Long G, Zhang C, Philip SY (2020) A comprehensive survey on graph neural networks. IEEE Trans Neural Netw Learn Syst 32(1):4–24
60. Seo Y, Defferrard M, Vandergheynst P, Bresson X (2018) Structured sequence modeling with graph convolutional recurrent networks. In: Proceedings of NIPS, pp. 362–373. Springer
61. Li Y, Yu R, Shahabi C, Liu Y (2018) Diffusion convolutional recurrent neural network: data-driven traffic forecasting. In: International conference on learning representations. https://openreview.net/forum?id=SJiHXGWAZ

62. Yu B, Yin H, Zhu Z (2017) Spatio-temporal graph convolutional networks: a deep learning framework for traffic forecasting. arXiv preprint arXiv:1709.04875
63. Yan S, Xiong Y, Lin D (2018) Spatial temporal graph convolutional networks for skeleton-based action recognition. In: Proceedings of AAAI, pp. 3482–3489
64. Yu B, Yin H, Zhu Z (2018) Spatio-temporal graph convolutional networks: a deep learning framework for traffic forecasting. In: Proceedings of the twenty-seventh international joint conference on artificial intelligence, IJCAI-18, pp. 3634–3640. International Joint Conferences on Artificial Intelligence Organization. https://doi.org/10.24963/ijcai.2018/505
65. Wu Z, Pan S, Long G, Jiang J, Zhang C (2019) Graph wavenet for deep spatial-temporal graph modeling. arXiv preprint arXiv:1906.00121
66. Kim R, So CH, Jeong M, Lee S, Kim J, Kang J (2019) Hats: a hierarchical graph attention network for stock movement prediction. arXiv preprint arXiv:1908.07999
67. Xiang S, Cheng D, Shang C, Zhang Y, Liang Y (2022) Temporal and heterogeneous graph neural network for financial time series prediction. In: Proceedings of the 31st ACM international conference on information & knowledge management, pp. 3584–3593
68. Gu J, Ye J, Uddin A, Wang G (2024) Dystage: dynamic graph representation learning for asset pricing via spatio-temporal attention and graph encodings. In: Proceedings of the 5th ACM international conference on AI in finance, pp. 388–396
69. Sefer E (2025) Drgat: predicting drug responses via diffusion-based graph attention network. J Comput Biol 32(3):330–350
70. Celik P, Sefer E (2025) Analyzing transaction graphs via motif-based graph representation learning for cryptocurrency price prediction. Comput Econ, 1–22
71. Eralp B, Sefer E (2024) Reference-free inferring of transcriptomic events in cancer cells on single-cell data. BMC Cancer 24(1):607
72. Tuncer T, Kaya U, Sefer E, Alacam O, Hoser T (2022) Asset price and direction prediction via deep 2d transformer and convolutional neural networks. In: Proceedings of the third ACM international conference on AI in finance, pp. 79–86
73. Abu-El-Haija S, Perozzi B, Kapoor A, Alipourfard N, Lerman K, Harutyunyan H, Ver Steeg G, Galstyan A (2019) Mixhop: Higher-order graph convolutional architectures via sparsified neighborhood mixing. In: International conference on machine learning, pp. 21–29. PMLR
74. Katznelson Y (2004) An introduction to harmonic analysis. Cambridge University Press, Cambridge
75. Aroussi R (2024) yfinance: Yahoo! Finance market data downloader. https://github.com/ranaroussi/yfinance. Accessed Jan 4
76. Cheung Y-W, Lai KS (1995) Lag order and critical values of the augmented Dickey-Fuller test. J Bus Econ Stat 13(3):277–280
77. Lempérière Y, Deremble C, Seager P, Potters M, Bouchaud J-P (2014) Two centuries of trend following. arXiv preprint arXiv:1404.3274
78. Cesa-Bianchi N, Lugosi G (2006) Prediction, learning, and games. Cambridge University Press, Cambridge
79. Li B, Hoi SC, Zhao P, Gopalkrishnan V (2013) Confidence weighted mean reversion strategy for online portfolio selection. ACM Trans Knowl Discov Data (TKDD) 7(1):1–38
80. Bergstra J, Yamins D, Cox DD et al (2013) Hyperopt: a python library for optimizing the hyperparameters of machine learning algorithms. In: Proceedings of the 12th python in science conference, vol. 13, pp. 20. Citeseer
81. Ngo VM, Nguyen HH, Nguyen P (2023) Does reinforcement learning outperform deep learning and traditional portfolio optimization models in frontier and developed financial markets? Res Int Bus Financ 65:101936
82. U.S. Department of the Treasury: Treasury Bill Rates Data. https://home.treasury.gov/resource-center/data-chart-center/interest-rates/TextView?type=daily_treasury_bill_rates, Accessed Jan 11, 2024
83. Paszke A, Gross S, Massa F, Lerer A, Bradbury J, Chanan G, Killeen T, Lin Z, Gimelshein N, Antiga L, et al (2019) Pytorch: an imperative style, high-performance deep learning library. Advances in neural information processing systems 32
84. Seabold S, Perktold J et al (2010) Statsmodels: econometric and statistical modeling with python. SciPy 7(1):92–96
85. Taleb NN (2020) Statistical consequences of fat tails: real world preasymptotics, epistemology, and applications. arXiv preprint arXiv:2001.10488
86. Xu M, Dai W, Liu C, Gao X, Lin W, Qi G-J, Xiong H (2020) Spatial-temporal transformer networks for traffic flow forecasting. arXiv preprint arXiv:2001.02908
87. Deihim A, Alonso E, Apostolopoulou D (2023) Sttre: a spatio-temporal transformer with relative embeddings for multivariate time series forecasting. Available at SSRN 4404879

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Yasin Uygun[1] · Emre Sefer[1]

✉ Emre Sefer
emre.sefer@ozyegin.edu.tr

[1] Artificial Intelligence and Data Engineering Department, Ozyegin University, Orman Sokak, 34794 Cekmekoy, Istanbul, Turkey